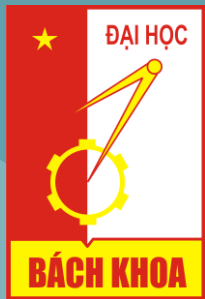


C PROGRAMMING INTRODUCTION



TUẦN 10: MẢNG

**Trường Công nghệ Thông tin và Truyền thông
Trường Đại học Bách Khoa Hà Nội**

Nội dung

- Khai báo mảng
- Khởi tạo mảng
- Truyền mảng cho hàm
- Mảng hai chiều

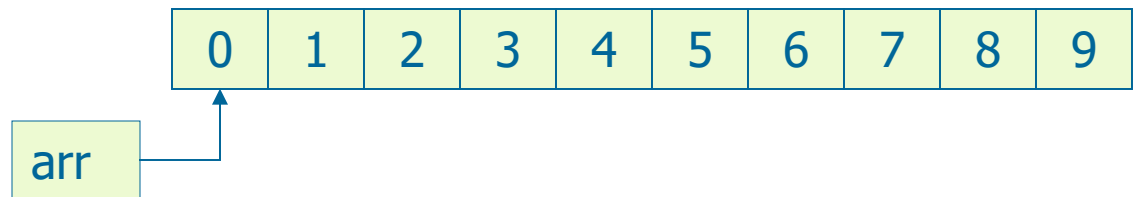
I. Định nghĩa Mảng

- Một khối gồm nhiều biến có cùng kiểu
- Mảng có thể được khai báo bằng kiểu bất kỳ
- Ví dụ: `int Arr[10]` là mảng gồm 10 phần tử số nguyên
- Ứng dụng:
 - Danh sách điểm của sinh viên
 - Chuỗi các số nhập vào bởi người dùng
 - Véc-tơ
 - Ma trận

Mảng trong bộ nhớ

- Một chuỗi các biến có kiểu dữ liệu xác định
- VD:

```
int arr[10];
```



- Phần tử thứ n của mảng `arr` được thể hiện bởi `arr[n-1]` **(0-based)**

Khai báo mảng

- `int arr[10];`
- Số phần tử của mảng cần phải biết trước khi biên dịch chương trình.

- Ví dụ:

```
int n = 10;
```

```
int s[n];
```

- Đoạn lệnh sau không báo lỗi biên dịch mà báo lỗi run-time:

```
int n;
```

```
int s[n];
```

Khởi tạo mảng

- Mảng có thể được khởi tạo khi khai báo
- Số lượng phần tử được khởi tạo không vượt quá kích thước mảng
- Các phần tử còn lại được khởi tạo bằng 0
- Kích thước mảng có thể được xác định dựa trên các giá trị khởi tạo
- Ví dụ:

```
int array1[8] = {2, 4, 6, 8, 10, 12, 14, 16};
```

```
int array2[] = {2, 4, 6, 8, 10, 12, 14, 16};
```

Bài thực hành 1: Khởi tạo với vòng for

- Chạy thử nghiệm chương trình
- Xác định kết quả của chương trình

```
#include <stdio.h>
```

```
#define ARRAY_SIZE 10
```

```
int main()
```

```
{
```

```
    int s[ARRAY_SIZE];    // array S has 10 elements
```

```
    int i;
```

```
    for ( i = 0; i < ARRAY_SIZE; i++ )
```

```
        s[i] = 2 + 2 * i;
```

```
    printf("Element \t Value\n");
```

```
    for ( i = 0; i < ARRAY_SIZE; i++ )
```

```
        printf("%d\t%d\n", i, s[i]);
```

```
    return 0;
```

```
}
```

Vào/ra dữ liệu với mảng

- Sử dụng vòng lặp for
- Tại mỗi bước lặp:
 - sử dụng hàm nhập dữ liệu như scanf
 - sử dụng hàm xuất dữ liệu như printf
- **Bài thực hành 2.**
 - A. Viết chương trình nhập dữ liệu cho lượng mưa của 12 tháng trong năm và in lượng mưa ra màn hình
 - B. Sử dụng kỹ thuật khởi tạo mảng với bộ dữ liệu bên phải và in lượng mưa của từng tháng ra màn hình.

month	rainfall (in mm)
1	40
2	45
3	95
4	130
5	220
6	210
7	185
8	135
9	80
10	40
11	45
12	30

table of rainfall

```
int RainFall[MONTHS] = {40,
45, 95, 130, 220, 210, 185,
135, 80, 40, 45, 30};
```

Bài thực hành 3: Chương trình dãy số - HUSTack

- Nhập một dãy số nguyên (nhiều nhất 10 phần tử)
- In dãy số đó theo thứ tự đảo ngược

Bài thực hành 4 - HUSTack

Viết chương trình nhập mảng chứa tối đa 20 số nguyên

- a) Tính tổng các số lẻ trong mảng
- b) Tìm số chẵn nhỏ nhất trong mảng.

Bài thực hành 5 - HUSTack

- Cho một mảng gồm các phần tử được nhập vào từ bàn phím.
- ***Tính tổng của các cực trị địa phương*** trong mảng (cực trị địa phương là phần tử có giá trị lớn hơn phần tử liền trước và liền sau nó)

II. Mảng là tham số của hàm

- Hàm cho phép nhận tham số là mảng
- Thường cần thêm tham số là kích thước mảng
- Ví dụ:
`int calc_sum(int arr[], int array_size);`
- Trong hàm **arr** được truy cập như bình thường: truy cập các phần tử qua chỉ số, ví dụ: `arr[1]`
- Thay đổi trong hàm tạo ra thay đổi trong mảng.

Bài thực hành 6. Tính tổng các phần tử của mảng

- Viết chương trình thử nghiệm hàm sau: trong hàm main, khai báo mảng, truyền mảng vào hàm, lấy kết quả từ hàm và hiển thị ra màn hình

```
int calc_sum(int arr[], int size)
{
    int i = 0;
    int sum = 0;

    for (i = 0; i < size; ++i)
        sum += arr[i];

    return sum;
}
```

Bài thực hành 7 - HUSTack

- Cài đặt một hàm nhận tham số là hai mảng, trả về 1 nếu hai mảng giống nhau, 0 nếu ngược lại
- Cú pháp của hàm:

```
int compare_arrays(const int arr1[],  
const int arr2[], int size);
```

- Viết chương trình sử dụng hàm này

Bài thực hành 8a

- Viết hàm sắp xếp các số nguyên trong mảng theo thứ tự giảm dần

- Cú pháp của hàm:

```
void DesSort (int a[], int n);
```

- Viết chương trình yêu cầu người dùng nhập vào mảng gồm tối đa 10 phần tử và thực hiện hàm trên

Lời giải

```
void DesSort (int a[], int n)
{
    int tmp;
    for (i = 0; i < n-1 ; i++)
        for (j = i+1; j < n; j++)
            if (a[i] < a [j]) {
                tmp = a[i];
                a[i] = a[j];
                a[j] = tmp;
            }
}
```

Bài thực hành 8b

- Viết hàm sắp xếp các số lẻ trong mảng theo thứ tự giảm dần, còn các số chẵn thì vẫn giữ nguyên vị trí
- Cú pháp của hàm như sau:

```
void OddSort (int a[], int n)
```

- Viết chương trình yêu cầu người dùng nhập vào mảng gồm tối đa 10 phần tử và thực hiện hàm trên

Bài thực hành 9 - HUSTack

- Cho một mảng số nguyên:
 - a) Đếm số phần tử có giá trị bằng 0
 - b) Tìm độ dài của chuỗi dài nhất gồm các phần tử có giá trị bằng 0
 - c) Đếm số lần xuất hiện của từng phần tử. Ví dụ: mảng 1, 2, 3, 1, 2, 3, 3, 4 thì số 1 xuất hiện 2 lần, số 2 xuất hiện 2 lần, số 3 xuất hiện 3 lần, số 4 xuất hiện 1 lần.

III. Mảng hai chiều

- Mảng của mảng:

```
int A[2][3] = { {1, 2, 3},  
                {4, 5, 6} };
```

1	2	3
4	5	6

- Mảng gồm 2 mảng số nguyên, mỗi mảng có 3 phần tử
- Truy cập: phần tử j của mảng i

A[i][j]

Bài thực hành 10

- Tạo ma trận 10x10 chứa bảng cửu chương.
- Nhập hai số m và n, tìm giá trị của $m \times n$ trong bảng cửu chương.



Bảng cửu chương

$1 \times 1 = 1$ $1 \times 2 = 2$ $1 \times 3 = 3$ $1 \times 4 = 4$ $1 \times 5 = 5$ $1 \times 6 = 6$ $1 \times 7 = 7$ $1 \times 8 = 8$ $1 \times 9 = 9$ $1 \times 10 = 10$	$2 \times 1 = 2$ $2 \times 2 = 4$ $2 \times 3 = 6$ $2 \times 4 = 8$ $2 \times 5 = 10$ $2 \times 6 = 12$ $2 \times 7 = 14$ $2 \times 8 = 16$ $2 \times 9 = 18$ $2 \times 10 = 20$	$3 \times 1 = 3$ $3 \times 2 = 6$ $3 \times 3 = 9$ $3 \times 4 = 12$ $3 \times 5 = 15$ $3 \times 6 = 18$ $3 \times 7 = 21$ $3 \times 8 = 24$ $3 \times 9 = 27$ $3 \times 10 = 30$	$4 \times 1 = 4$ $4 \times 2 = 8$ $4 \times 3 = 12$ $4 \times 4 = 16$ $4 \times 5 = 20$ $4 \times 6 = 24$ $4 \times 7 = 28$ $4 \times 8 = 32$ $4 \times 9 = 36$ $4 \times 10 = 40$	$5 \times 1 = 5$ $5 \times 2 = 10$ $5 \times 3 = 15$ $5 \times 4 = 20$ $5 \times 5 = 25$ $5 \times 6 = 30$ $5 \times 7 = 35$ $5 \times 8 = 40$ $5 \times 9 = 45$ $5 \times 10 = 50$
$6 \times 1 = 6$ $6 \times 2 = 12$ $6 \times 3 = 18$ $6 \times 4 = 24$ $6 \times 5 = 30$ $6 \times 6 = 36$ $6 \times 7 = 42$ $6 \times 8 = 48$ $6 \times 9 = 54$ $6 \times 10 = 60$	$7 \times 1 = 7$ $7 \times 2 = 14$ $7 \times 3 = 21$ $7 \times 4 = 28$ $7 \times 5 = 35$ $7 \times 6 = 42$ $7 \times 7 = 49$ $7 \times 8 = 56$ $7 \times 9 = 63$ $7 \times 10 = 70$	$8 \times 1 = 8$ $8 \times 2 = 16$ $8 \times 3 = 24$ $8 \times 4 = 32$ $8 \times 5 = 40$ $8 \times 6 = 48$ $8 \times 7 = 56$ $8 \times 8 = 64$ $8 \times 9 = 72$ $8 \times 10 = 80$	$9 \times 1 = 9$ $9 \times 2 = 18$ $9 \times 3 = 27$ $9 \times 4 = 36$ $9 \times 5 = 45$ $9 \times 6 = 54$ $9 \times 7 = 63$ $9 \times 8 = 72$ $9 \times 9 = 81$ $9 \times 10 = 90$	$10 \times 1 = 10$ $10 \times 2 = 20$ $10 \times 3 = 30$ $10 \times 4 = 40$ $10 \times 5 = 50$ $10 \times 6 = 60$ $10 \times 7 = 70$ $10 \times 8 = 80$ $10 \times 9 = 90$ $10 \times 10 = 100$

Bài thực hành 11

- Viết chương trình khai báo 3 ma trận A, B, C kích thước 3x3 chứa số nguyên; khởi tạo A và B
- Nhập hai ma trận A và B từ bàn phím
- Tính kết quả của phép nhân ma trận và lưu vào C: $C = A * B$
- In kết quả ra màn hình

Bài thực hành 12 - HUSTack

- Nhập mảng N phần tử số nguyên từ người dùng
- Kiểm tra tính đối xứng của mảng

Bài thực hành 13

- Viết hàm đảo ngược mảng.

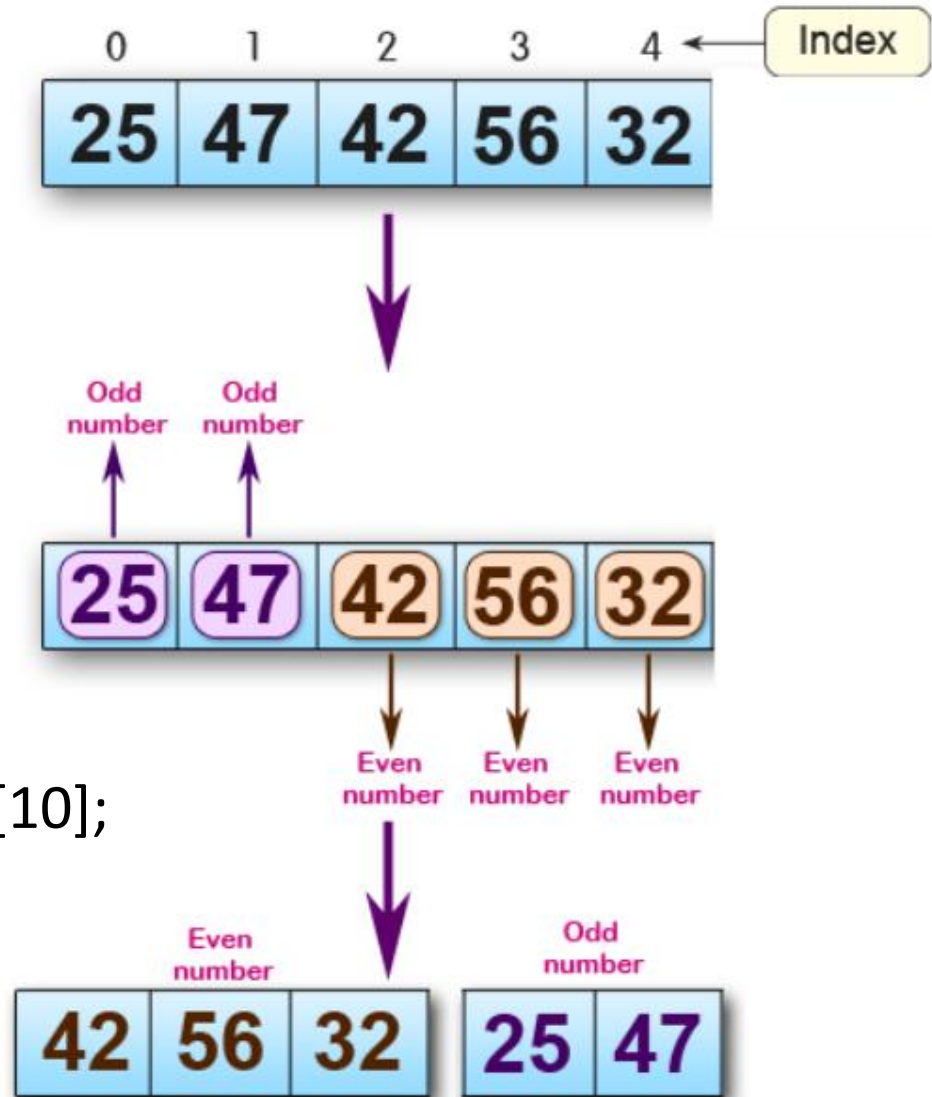
```
void reverse(float a[], int size);
```

- Sử dụng hàm này trong chương trình với mảng số thực được nhập từ bàn phím.

Bài thực hành 14

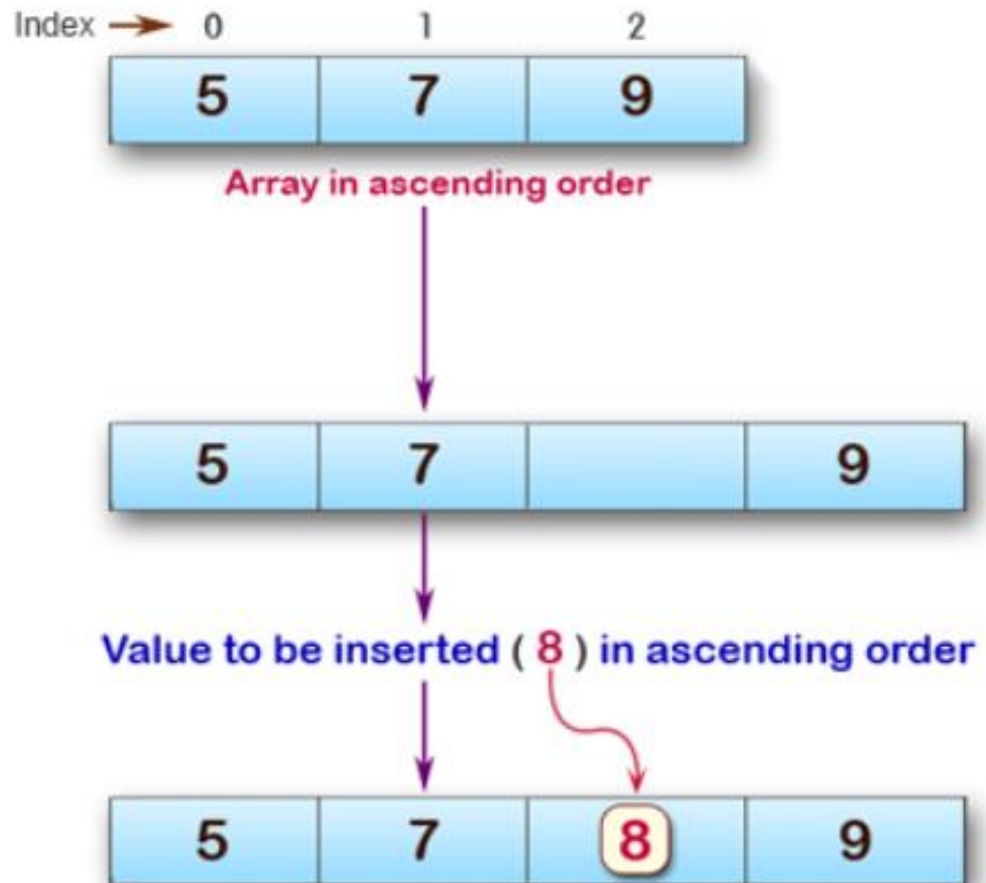
- Viết chương trình phân tách các số chẵn và số lẻ của một mảng ra hai mảng khác

```
int arr1[10], arr2[10], arr3[10];
```



Bài thực hành 15 - HUSTack

- Viết chương trình chèn một phần tử mới vào vị trí chính xác trong một mảng
- Biết mảng này đã sắp xếp theo thứ tự tăng dần



Bài thực hành 16 - HUSTack

- Viết chương trình tính định thức của ma trận 3x3

$$A = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$$



$$A = a(ei - fh) - b(di - fg) + c(dh - eg)$$



$$\begin{matrix} a_x & & b_x & & c_x \\ \begin{bmatrix} e & f \\ h & i \end{bmatrix} & - & \begin{bmatrix} d & f \\ g & i \end{bmatrix} & + & \begin{bmatrix} d & e \\ g & h \end{bmatrix} \end{matrix}$$



$$A = a \begin{bmatrix} e & f \\ h & i \end{bmatrix} - b \begin{bmatrix} d & f \\ g & i \end{bmatrix} + c \begin{bmatrix} d & e \\ g & h \end{bmatrix}$$

Bài thực hành 17

Kiểm tra ma trận thưa

- Một ma trận thưa là ma trận có nhiều phần tử bằng không hơn các phần tử khác không.
- Input:
 - 2 số nguyên n và m biểu diễn số hàng và số cột của ma trận số nguyên (tối đa 10).
 - Mỗi dòng biểu diễn các phần tử của một hàng của ma trận, các phần tử ngăn cách nhau bằng dấu cách.
- Output: “YES” nếu là ma trận thưa và “NO” nếu không phải.

0	0
1	0



0	0
1	0

There are 3 number of zeros in the matrix



The given matrix is sparse matrix

Bài thực hành 18

Tìm phần tử đa số của một mảng số nguyên dương

- Phần tử đa số trong mảng $A[]$ có kích thước n là phần tử xuất hiện nhiều hơn $n/2$ lần (và do đó chỉ có nhiều nhất một phần tử như vậy).
- Input:
 - Số nguyên N : số phần tử của mảng
 - Các phần tử của mảng
- Output:
 - Giá trị của phần tử này
 - -1 nếu không có phần tử nào như vậy

Bài thực hành 19

- Viết một chương trình bằng C để in một ma trận theo dạng xoắn ốc.
- Input:
 - 2 số nguyên n và m biểu diễn số hàng và số cột của ma trận số nguyên (tối đa 10).
 - Mỗi dòng biểu diễn các phần tử của một hàng của ma trận, các phần tử ngăn cách nhau bằng dấu cách.
- Output: các phần tử theo thứ tự duyệt xoắn ốc bắt đầu từ góc trên cùng bên trái, các phần tử này đều hiển thị trên một dòng, ngăn cách bởi dấu cách

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

Matrix in spiral form

